Python Based PCAP Security Investigation and Threat Detection System

MINI PROJECT FINAL REPORT

Information Security Lab

Student Name(s)	Abhiyansh Kapoor Saniya Taneja Tanvi Kavi
Roll No(s)	240953614 240953612 240953318
Programme / Semester / Section	B.Tech / 5th Semester / CCE-C
Course	Information Security Lab
Course Code	ICT 3141
Faculty / Guide	Mr. Abhishek A K

Academic Year: 2026–2027

Python Based PCAP Security Investigation and Threat Detection System

Abhiyansh Kapoor
240953614
Manipal Institute of Technology
MAHE, Manipal, India

Saniya Taneja
240953612
Manipal Institute of Technology
MAHE, Manipal, India

Tanvi Kavi
240953318
Manipal Institute of Technology
MAHE, Manipal, India

***Abstract*—Manual review of packet captures is slow and may miss repeated requests whose individual features look ordinary. This project implements a Python workflow that extracts HTTP requests from PCAP and PCAPNG files with PyShark and TShark, scores request properties inspired by prior malware traffic research, and groups related requests into investigation cases. The single-request baseline considers protocol version, headers, User-Agent, Host, and body characteristics. The case layer adds repetition, timing, POST and GET check-in sequences, and recognizable web-attack payloads. Findings are saved in SQLite and presented in an HTML report with attack assessments, mitigation suggestions, source provenance, and Wireshark frame numbers. Evaluation used labelled generated traffic, four published packet captures, and a newly recorded local HTTPS capture whose TLS session secrets permit authorized decoding. On the labelled infected-host fixture, request-level recall rose from 25 of 40 malicious requests to 40 of 40 after correlation. These controlled results are not an estimate of accuracy on arbitrary live traffic. The published captures yielded four distinct evidence-backed case types; the local HTTPS exercise yielded five simulated cases from 137 requests. The system assists investigation rather than blocking traffic, and encrypted sessions remain opaque without their keys.**

***Index Terms*—PCAP analysis, HTTP anomaly detection, behavior correlation, network forensics, HTTPS decryption**

1. Introduction

Background and problem. Packet captures preserve network exchanges but are difficult to review manually when many web requests appear. A useful investigator needs both clues within a request and behavior spread across several requests. Missing common headers or unusual payload bytes can be helpful, yet one request rarely establishes an attack [1].

Research gap and objectives. The original feature study focuses on differences between malware and browser HTTP traffic [1]. This project converts a small subset of those ideas into transparent rules and then adds source-to-destination correlation, packet references, and an analyst report. Its goals are to extract HTTP features, raise reviewable cases, persist the evidence, and compare single-request rules with correlation on labelled test traffic.

Scope. The analysis works on cleartext HTTP and on HTTPS only when the capture includes usable session secrets. It reports attempted web attacks from request payloads; a request alone does not prove server compromise. Section 2 reviews prior methods, Section 3 describes the implementation, Section 4 presents measured results, and Section 5 states limitations and future work.

2. Related Work

Białczak and Mazurczyk examined request structure, header values, and payload properties in malware and browser HTTP traffic [1]. Their results motivate our feature set, but our equal-weight score and threshold are our own simple baseline, not a reproduction of their experimental model or reported accuracy. BAYWATCH shows why repeated callbacks and timing can reveal behavior that isolated events miss; it also warns that legitimate applications beacon [2]. Hfinger groups malware HTTP requests using request fingerprints [3]. We do not implement Hfinger fingerprinting. Instead, our modest extension groups requests by source and HTTP Host and records repeated paths, timing, and POST/GET sequences. This makes the output easy to inspect, while leaving open the possibility of false positives from updates and automated tools.

3. Methodology

System architecture. The pipeline is capture file → PyShark/TShark parsing → feature extraction → request score and behavior correlation → SQLite storage → HTML case report. The parser reads only decoded HTTP requests and preserves frame number, time, source, destination, method, URI, headers, and available body bytes [4]. The report links each case back to the named PCAP and its Wireshark frame numbers.

Dataset and provenance. Two generated captures provide known destination labels: 414 normal-browsing requests and 561 infected-host requests, 40 of the latter labelled malicious by the fixture generator. Four published captures supply varied examples: a 2026 FormBook traffic-analysis exercise [6], a CodePath web-attack lab capture [7], and separate SQL injection and XSS test captures from an IDS benchmark collection [8]. The local HTTPS demonstration records 137 new requests to 127.0.0.1:8443. Different .localhost Host labels separate its five scenarios; all connections remain on one laptop.

Preprocessing and features. For each decoded request, the program counts headers, checks HTTP version and selected header presence, tests whether Host is an IP address, and measures body length, byte entropy, and non-ASCII share.

The score is the number of triggered indicators. A score of at least three creates a single-request feature alert. These clues are deliberately simple and are not treated as proof of malware.

Correlation and case rules. Requests are grouped by source IP and HTTP Host. The case layer considers repeated alerts, repeated URI paths, regular inter-request gaps, and a body-bearing POST plus long-query GET check-in sequence. A burst requires at least 100 requests, at least 20 feature alerts, and a rate of at least 10 requests per second within 30 seconds. Separate request-pattern checks recognize directory traversal paths, UNION SELECT queries, and script tags, including encoded query values. Each case includes a risk score, severity, evidence list, attack assessment, and suggested mitigation. These labels describe observed patterns, not confirmed compromise.

HTTPS experimental setup. A Python client sends harmless requests to a local TLS server while Npcap and TShark record loopback traffic. The client logs TLS session secrets and editcap embeds them in the PCAPNG. Wireshark can then decode this particular capture [5]. This mechanism does not decrypt unrelated HTTPS traffic. The local scenarios contain 120 rapid POSTs, three traversal payloads, three SQL injection payloads, three XSS payloads, seven repeated check-ins, and one normal GET.

Evaluation metrics. For the generated labelled fixture, a true positive is a request to a fixture-labelled malicious destination that the method marks for review. We report request-level true positives, false positives, false negatives, precision, recall, and F1. For published and live captures without complete ground truth, we report counts and packet evidence only. The eight automated unit tests check case labels, threshold behavior, and database compatibility.

4. Results and Discussion

Controlled comparison. Table I compares methods on the generated infected-host capture. The feature score detected 25 of 40 labelled malicious requests. Correlation linked all 40 to medium or high cases; both methods marked one other request. The 26 baseline alerts are individual requests, whereas the five correlated alerts are analyst cases, so their review counts have different units. The separate 414-request normal fixture produced one alert under each method; with no labelled malicious requests, it cannot provide a useful recall estimate. The fixture labels were built into the test data, so these results cannot be generalized to field malware traffic.

TABLE I. Generated infected-host fixture: request-level comparison

Metric	Feature rules	Correlated
True positives	25	40
False positives	1	1
False negatives	15	0
Precision	0.96	0.98
Recall	0.62	1.00
F1	0.76	0.99

Published captures. The combined published-data report examines 186 HTTP requests. In the FormBook exercise capture, 128 requests form a repeated POST/GET sequence across 14 destinations from one source; the publisher's exercise labels the traffic as FormBook [6]. A separate lab capture contains a traversal request at frame 74 and a response at frame 76 with the requested file contents [7]. The SQL injection capture contains a UNION SELECT request, and the XSS capture contains a query value that decodes to a script tag [8]. The last two establish attempts, not successful compromise. The report consolidates these into four evidence-backed case types rather than counting each related destination as a different attack type.

Local HTTPS capture. The fresh PCAPNG contains 137 decoded HTTPS requests and five high-priority cases: an attempted one-source DoS flood (120 requests in about half a second), traversal (3), SQL injection (3), XSS (3), and simulated C2-style check-ins (7). One normal GET remains outside the cases. All 136 case-linked request frames appear in the report, and the flood server returned HTTP 200 responses; no outage or distributed DDoS was demonstrated. The results confirm that known local TLS secrets make this capture inspectable, not that arbitrary encrypted traffic is visible.

TABLE II. Capture sources and observed case evidence

Capture	Req.	Finding
FormBook exercise	176	128 C2-pattern requests
CodePath lab	8	Traversal and response
SQL test	1	UNION SELECT attempt
XSS test	1	Script query attempt
Local HTTPS	137	5 cases; 136 linked

Interpretation and limits. The source collections were selected to demonstrate patterns, not sampled to estimate prevalence. Header anomalies can occur in benign scripts, and encoded payload rules can miss new encodings or match harmless text. The small local web server has no database or browser execution path, so its SQL injection and XSS labels refer only to submitted payloads. The tool suggests mitigations but does not enforce a firewall rule. More diverse independently labelled traffic is needed before reporting operational precision or recall.

5. Conclusion and Future Work

The project provides a reproducible path from saved PCAP or PCAPNG to scored requests, grouped cases, SQLite records, and a frame-linked HTML investigation report. The controlled fixture shows the added value of grouping repeated requests, and the published and local captures demonstrate four and five distinct case types respectively. The strongest result is traceability: an analyst can open the exact packet behind a finding. Future work should test a larger independent benign and malicious corpus, tune thresholds against false alerts, and obtain authorized HTTPS visibility through endpoint key logs or server-side telemetry. Pattern

labels should continue to distinguish attempts from confirmed outcomes.

Team responsibilities. Abhiyansh Kapoor handled the local HTTPS capture with TLS session-key decoding and the case report with attack assessments, mitigation guidance, and Wireshark frame links. Saniya Taneja handled published traffic sourcing and provenance, HTTP feature scoring, behavior correlation, and real-data case analysis. Tanvi Kavi handled the report UI, SQLite case storage, generated test fixtures, and verification tests. All three are listed as authors and reviewed the final submission.

References

[1] P. Białczak and W. Mazurczyk, "Characterizing Anomalies in Malware-Generated HTTP Traffic," Security and Communication Networks, vol. 2020, Art. no. 8848863, 2020, doi: 10.1155/2020/8848863.

[2] X. Hu et al., "BAYWATCH: Robust beaconing detection to identify infected hosts in large-scale enterprise networks," in Proc. IEEE/IFIP DSN, 2016. Available: research.ibm.com/publications/baywatch-robust-beaconing-detection-to-identify-infected-hosts-in-large-scale-enterprise-networks.

[3] P. Białczak and W. Mazurczyk, "Hfinger: Malware HTTP Request Fingerprinting," Entropy, vol. 23, no. 5, Art. no. 507, 2021, doi: 10.3390/e23050507.

[4] KimiNewt, "PyShark FileCapture source," GitHub. Available: github.com/KimiNewt/pyshark/blob/master/src/pyshark/capture/file_capture.py. Accessed: Nov. 2026.

[5] Wireshark, "TLS," Wireshark Wiki. Available: wiki.wireshark.org/TLS. Accessed: Nov. 2026.

[6] Malware-Traffic-Analysis.net, "2026-08-09 traffic analysis exercise: First to Last," 2026. Available: malware-traffic-analysis.net/2026/08/09/index.html.

[7] CodePath, "Snort NIDS Lab and web-attack.pcap," GitHub. Available: github.com/codepath/opencyber-snort-lab. Accessed: Nov. 2026.

[8] Trepverterless, "IDS-Benchmark-pcaps," GitHub. Available: github.com/Trepverterless/IDS-Benchmark-pcaps. Accessed: Nov. 2026.